ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Интеграционное тестирование

Модуль 6 · Лекция 1 · Границы уровня, Testcontainers, тестирование базы данных

Ирина Габанова · Lekton

4 курс

6 ноября · Четверг 13:30–15:00

МОДУЛЬ 6 · ЛЕКЦИЯ 1

План лекции

01 Где мы в курсе: unit → API → интеграционные → E2E

02 Интеграционный тест в микросервисах: определение и границы уровня

03 Managed vs unmanaged зависимости: что мокировать, а что поднимать

04 Почему H2 и in-memory БД — антипаттерн

05 Testcontainers: устройство, жизненный цикл, первый тест на PostgreSQL

06 Интерактив: «Unit или интеграционный?»

Модуль 6 в карте курса

Модуль 4: Unit-тесты
Логика в изоляции, всё внешнее замокано.
Сотни тестов, миллисекунды на тест.

Модуль 5: API-тесты
Контракт между сервисами, HTTP-стык.
Десятки тестов на эндпоинт, минуты.

Модуль 6: Интеграционные
Сервис + его реальная инфраструктура (БД, очередь).
Единицы-десятки тестов, секунды на тест.

Модуль 7: E2E
Вся цепочка сквозным сценарием.
Считанные сценарии, минуты на тест.

Ключевое различие: в API-тесте вы снаружи — отправили запрос, проверили ответ. В интеграционном тесте вы идёте внутрь — проверяете реальное состояние: легла ли запись в таблицу, применилась ли миграция, ушло ли сообщение в очередь.

Зачем нужен отдельный интеграционный уровень

Unit-тесты с моками проверяют логику, но не проверяют **стык**: SQL, маппинг, транзакции, протокол брокера.

Класс дефектов, невидимый для unit-тестов: ошибка в SQL, несовпадение схемы, неприменённая миграция, неверная сериализация сообщения.

Дефект на интеграционном уровне в 10 раз дороже unit-уровня, но в 10 раз дешевле E2E (правило 1:10:100).

Микросервисы усиливают риск: стыков больше, чем в монолите — каждый сервис имеет собственный persistence-слой.

Пример из СМП: в Authorization лимиты проверяются одним SQL-запросом — атомарным upsert в таблицу `limit_usage`. Вся корректность авторизации по лимиту зависит от того, верно ли написано условие в WHERE этого запроса. Unit-тест с моком репозитория проверить его не может в принципе.

МОДУЛЬ 6 · ЛЕКЦИЯ 1

Границы уровня

МОДУЛЬ 6 · ЛЕКЦИЯ 1

Что такое интеграционный тест в микросервисах

Классика: тест взаимодействия модулей, собранных вместе — сборка подсистемы. Стратегии: сверху вниз, снизу вверх, «большой взрыв».

В микросервисах смысл сдвигается (TJM, гл. 5): проверяем, что **подсистема сервиса корректно общается с внешним компонентом** — БД, брокером, другим сервисом.

Цель — проверить коммуникацию, а не сам внешний компонент. Правильность PostgreSQL тестирует PostgreSQL Global Development Group.

Идеальный сценарий — без тестовых дублёров на внешних компонентах. Мок базы данных в интеграционном тесте — оксюморон.

Покрываем только базовые пути успеха и ошибки интеграции, а не всю логику — комбинаторика остаётся на unit-уровне.

Критерий: если дефект можно починить только патчем вендору — тест чужой; если правкой нашего репозитория — наш.

Managed и unmanaged зависимости

Managed (управляемые): зависимость, которой владеет только наше приложение — его база данных → **поднимаем как есть**, мок запрещён.

Unmanaged (неуправляемые): внешний сервис, SMTP, чужой API → **подменяем** (mock, stub, service virtualization).

Критерий: можем ли мы «сломать» её состояние для других? Если нет — managed.

Тонкость: критерий — не «внутренняя или внешняя система», а именно «разделяемое ли состояние».

В СМП: PostgreSQL и RabbitMQ сервиса — managed; BIN-сервис и внешний процессинг — unmanaged.

Граница между unit- и интеграционным тестом

Один и тот же класс (например, `CardRepository`) тестируется дважды — по-разному. Это **НЕ дублирование**: разные объекты проверки.

Unit-тест сервиса
Репозиторий замокан, проверяем логику сервиса.
Объект: бизнес-логика — правильное ли решение при таких данных.

Интеграционный тест репозитория
Реальная БД, проверяем SQL и маппинг.
Объект: сам репозиторий как код стыка — верен ли SQL, правильно ли замаплены колонки.

Признак интеграционного теста: тест падает, если стык с реальным компонентом сломан, даже когда логика верна.
Обратный вопрос: если логика сломана, а стык идеален — упадёт ли? Интеграционный тест не обязан падать в этом случае — для этого есть unit-уровень.

МОДУЛЬ 6 · ЛЕКЦИЯ 1

База данных в тестах

Почему H2 — антипаттерн

Соблазн: подменить PostgreSQL на in-memory H2/SQLite — быстро, без Docker. Проблема (Khorikov, гл. 10): in-memory СУБД **несовместима функционально** с боевой.

Другой диалект SQL: запрос с `ILIKE`, с `RETURNING`, с `jsonb`-операторами работает на PostgreSQL и не существует на H2.

Другие типы: `BIGSERIAL`, `NUMERIC` с фиксированной точностью, `timestamptz` — у каждого аналога свои граничные различия.

Другое поведение ограничений: где-то регистр сравнения строк другой, где-то уникальный индекс обрабатывает NULL иначе.

Режимы совместимости не спасают: это подмножество, а не эквивалент.

Последствие: false positives (тест зелёный, прод красный) и false negatives.
Правило: в тестах — та же СУБД и та же мажорная версия, что в проде; сегодня это дёшево благодаря контейнерам.

Схема БД — это код: миграции

Схема в Git вместе с кодом — единый источник истины (Khorikov, 10.1). Изменяется только через коммиты, ревьюится как код.

Migration-based подход: версионируемые скрипты Flyway/Liquibase. State-based (эталонная БД + diff-скрипты) — антипаттерн.

Reference data — часть схемы: справочники, без которых приложение не работает, тоже в миграциях.

Отдельный инстанс БД на разработчика — никаких общих тестовых баз. Контейнеры закрывают этот пункт изящно.

Тест поднимает «боевой» путь: контейнер → миграции → схема как в проде. *Сам факт того, что тест стартует на пустом контейнере и накатывает все миграции с нуля, уже является тестом миграций.*

В СМП: Card Management — каталог `db/migration` с последовательностью `V5.1__init_cards_table.sql`, `V5.2__init_bin_issuers_table.sql` и далее. Каждое изменение схемы — отдельный маленький скрипт с понятным именем.

МОДУЛЬ 6 · ЛЕКЦИЯ 1

Testcontainers

Testcontainers: как это устроено

Библиотека для Java: управляет Docker-контейнерами из тестового кода.

1	2	3	4	5
pull образа (кэширован)	старт контейнера + случайный порт	wait strategy: ожидание готовности	инъекция URL в тест	остановка и удаление (Ryuk)

Типизированные модули: `PostgreSQLContainer`, `RabbitMQContainer` — с готовыми wait-стратегиями.

`GenericContainer` — для любого образа без модуля, с явной настройкой портов и ожидания.

Готовность ≠ старт: контейнер «up» не значит «PostgreSQL принимает соединения» — wait strategy решает.
Требование: Docker на машине разработчика и на CI-агенте.

Жизненный цикл контейнера: per-class, per-method, singleton

Per-class (static @Container)
Контейнер один на класс: старт ~10 сек один раз, тесты делят базу.
Рекомендация курса: баланс скорости и изоляции.

Per-method (@Container)
Контейнер на каждый метод: чистая база всегда, но +10 сек × N тестов.
Для особо грязных сценариев.

Singleton Container
Ручной static старт без JUnit-аннотаций — один контейнер на весь набор классов.
Когда классов много; нужна дисциплина очистки.

Правило курса: static per-class + очистка данных в начале теста; singleton — когда классов много.

Изоляция достигается очисткой данных, а не пересозданием контейнера.

Экономика: старт PostgreSQL — ~10 сек. Per-method на 50 тестов = 8+ минут только на подъёмы. Per-class на 10 классов = 100 сек. Singleton = 10 сек на весь набор.

Первый интеграционный тест: PostgreSQL в Testcontainers

```
@DataJpaTest
@Testcontainers
@AutoConfigureTestDatabase(
    replace = AutoConfigureTestDatabase.Replace.NONE)
@Import(CardRepositoryPersistenceAdapter.class)
class CardRepositoryIT {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired
    private CardRepository repository;
```

```
    @Test
    void shouldCreateAndFindCard() {
        // Arrange
        Card card = new Card(UUID.randomUUID(), PAN, BIN,
            "IVAN IVANOV", YearMonth.now().plusYears(3),
            CardStatus.ACTIVE, "643",
            new BigDecimal("10000"), new BigDecimal("300000"),
            new BigDecimal("1000000"), "issuer-1", Instant.now());

        // Act
        Card saved = repository.create(card);
        Optional<Card> found = repository.findByPan(PAN);

        // Assert
        assertThat(found).isPresent();

assertThat(found.get().status()).isEqualTo(CardStatus.ACTIVE);
        assertThat(found.get().dailyLimit())
            .isEqualByComparingTo("10000");
    }
}
```

`@Testcontainers` — расширение JUnit 5, управляет жизненным циклом контейнеров.

`@Container` + static — режим per-class: один контейнер на класс.

`@ServiceConnection` — мост между контейнером и конфигурацией приложения.

Что проверять в persistence-тестах: чек-лист

Round-trip: сохранил → прочитал → поля совпали (включая типы, точность, NULL).

Ограничения: unique, not-null, FK — нарушение даёт ожидаемое исключение.

Миграции: свежая база + все миграции = код работает (схема не «модельная»).

Запросы с диалектом: фильтры, сортировки, пагинация, JSONB — на реальной СУБД.

Транзакционность: сбой посередине операции → состояние согласовано (rollback).

НЕ проверяем: бизнес-правила и комбинаторику — это unit-уровень.

В СМП: сохранение карты в `cards` и чтение обратно ловит маппинг `dailyLimit` в `daily_limit`; повторная вставка PAN ловит `uk_cards_pan`; атомарный `INSERT ... ON CONFLICT` по `limit_usage` тестирует диалект PostgreSQL.

МОДУЛЬ 6 · ЛЕКЦИЯ 1

Интерактив

Интерактив: «Unit или интеграционный?»

A: `AuthServiceTest` — мок репозитория, проверка decline по лимиту

B: `CardRepositoryIT` — H2 в режиме PostgreSQL, round-trip сохранения карты

C: `LimitServiceIT` — Testcontainers, PostgreSQL, проверка 15 классов эквивалентности суммы

D: `BinClientTest` — мок HTTP-клиента, проверка маппинга ответа BIN-сервиса

E: `CardRepositoryIT` — Testcontainers, проверка unique-ограничения на PAN

Задание: классифицируйте тест (unit / IT) и найдите проблему, если она есть. Аргументируйте через сегодняшние правила: managed-зависимости, объект проверки, чек-лист persistence-тестов.

Выводы и анонс

Интеграционный тест в микросервисе проверяет **стык с внешним компонентом**, а не сборку внутренних модулей.

Managed — поднимаем как есть; **unmanaged** — подменяем.

H2/in-memory — антипаттерн: другая СУБД, ложная уверенность.

Схема — код: миграции в Git, reference data в миграциях.

Testcontainers: реальная СУБД в тесте за одну аннотацию; изоляция — очисткой данных, не пересозданием контейнера.

Лекция 2: RabbitMQ, асинхронность, Awaitility, тестовые данные и flaky-тесты.

Проверка на понимание: представьте новый сервис антифрод в СМП — какие зависимости managed, какие unmanaged? Сколько контейнеров понадобится в его интеграционных тестах? Что будет в чек-листе его persistence-тестов?

Вопросы?

Интеграционное тестирование · Testcontainers · PostgreSQL